

A project on
“BASIC BANNER GRABBING”

Submitted by

NAME	REGISTRATION NUMBER
ANKUR SINGH	12522907
AREENA	12523061
MOHAN KUMAR JENA	12526672
YUKTI VERMA	12525095

Under the supervision of

Prof. **Manish Singh**, *CISSP*

Assistant Professor, Cyber Security & Networks

Cert Member, Ex-Nvidia Drivex Security Engineer

Submitted to

Lovely Professional University, Phagwara, Punjab

in partial fulfilments for the award of degree of “Masters of Computer Applications(hons)in Cyber Security”

Batch: 2025-2027

DECLARATION

We,(Ankur, Areena, Mohan, Yukti), hereby declare that the Project Report titled **“Banner Grabbing”** submitted to Lovely Professional University, Phagwara , Punjab in partial fulfillment of the requirement for the award of the degree “MCA(hons)in Cyber Security” is our original work. The findings and conclusions are based on the work carried out by us during the internship under the guidance of our internship mentor. This project has not been submitted, either in whole or in part, to any other university or institution for the award of any degree, diploma, or certificate.

Date: 7/07/2026

ACKNOWLEDGMENT

We express our sincere gratitude to Lovely Professional University for providing us with the opportunity to undertake this internship project.

We are especially thankful to our internship mentor for their valuable guidance, continuous support, and constructive suggestions throughout the completion of the project titled "Basic Banner Grabbing." Their encouragement and technical expertise helped us successfully complete this work.

We also extend our heartfelt thanks to our faculty members, friends, and team members for their cooperation, motivation, and assistance during the project. Their support played an important role in the successful completion of this internship report.

Finally, we would like to thank everyone who directly or indirectly contributed to the successful completion of this project.

CONTENT

Sr. no.	CONTENT	Page No.
<u>1.</u>	PROJECT TITLE	1
<u>2.</u>	DECLARATION	2
<u>3.</u>	ACKNOWLEDGMENT	3
<u>4.</u>	CHAPTER 1: INTRODUCTION	5-10
<u>5.</u>	CHAPTER 2: LITERATURE REVIEW	11-15
<u>6.</u>	CHAPTER 3: PROBLEM STATEMENT	16-18
<u>7.</u>	CHAPTER 4: RESEARCH METHODOLOGY	19-25
<u>8.</u>	CHAPTER 5: DATA ANALYSIS AND INTERPRETATION	26-31
<u>9.</u>	CHAPTER 6: DISCUSSIONS AND RESULTS	32-36
<u>10.</u>	CHAPTER 7: CONCLUSION	37-39
<u>11.</u>	CHAPTER 8: RECOMMENDATIONS AND SUGGESTIONS	40-43
<u>12.</u>	REFERENCES/BIBLIOGRAPHY	44-45
<u>13.</u>	APPENDICES/ANNEXURES	46-58

CHAPTER 1

1.1 Introduction

With the rapid growth of computer networks and internet-connected systems, identifying the services running on remote machines has become an essential task in cybersecurity. Before performing vulnerability assessment or penetration testing, security professionals first gather information about the target system. This initial phase is known as reconnaissance or enumeration. One of the simplest and most effective techniques used during this phase is **banner grabbing**.

Banner grabbing is a technique that establishes a TCP connection with a network service and collects the information returned by the server. Many services such as FTP, SSH, HTTP, SMTP, and Telnet automatically send identification details after a successful connection. These details, known as banners, often include the service name, software version, operating system information, and protocol details.

The **Basic Banner Grabbing Tool** developed in this project is implemented using Python's Socket Library. The application connects to user-specified ports on a target IP address or hostname and attempts to retrieve service banners. If the service does not automatically return a banner, the tool sends a simple HTTP GET request to trigger a response. The collected information is displayed through a Flask-based web interface and can also be downloaded as a text report.

The project introduces students to TCP socket programming, network communication, service identification, and the enumeration phase of cybersecurity while maintaining a simple and beginner-friendly implementation.

1.2 Overview of the Topic

Banner grabbing is a reconnaissance technique used to identify services running on network hosts by retrieving identification information exposed through open TCP ports.

Every network service communicates through one or more ports. When a client establishes a TCP connection with certain services, the server often responds with information describing itself. This response is called a banner.

Examples include:

- FTP servers displaying software name and version.
- SSH servers returning protocol version information.
- Web servers sending HTTP headers that reveal server software.
- Mail servers providing SMTP identification.

Unlike simple port scanning, which only determines whether a port is open, banner grabbing attempts to identify the exact application running on that port.

In this project, the user enters:

- Target hostname or IP address
- List of ports to scan

The application then:

- Creates a TCP socket.
- Connects to each specified port.
- Waits for banner information.
- Sends an HTTP GET request if no banner is received.
- Displays scan results.
- Generates a downloadable report.

The project demonstrates practical implementation of service identification using Python Socket Programming and provides a simple web interface using Flask for improved usability.

1.3 Problem Statement

Many beginners in cybersecurity understand the concept of network scanning but lack practical experience in implementing socket-based communication. Existing tools such as Nmap are powerful but often hide the underlying networking concepts behind automated features.

The objective of this project is to develop a simple Banner Grabbing Tool that demonstrates how TCP socket programming can be used to communicate directly with remote services and retrieve service identification information.

The project should:

- Connect to specified TCP ports.
- Retrieve service banners.
- Display software information whenever available.
- Handle connection failures gracefully.
- Generate a downloadable report.
- Demonstrate the enumeration phase of cybersecurity.

1.4 Objectives

The primary objectives of this project are:

- To understand TCP socket programming using Python.
- To establish network connections with remote hosts.
- To retrieve service banners from open ports.
- To identify running network services.
- To determine available software version information.
- To demonstrate the enumeration phase of penetration testing.
- To build a simple web interface using Flask.

- To generate downloadable scan reports.
- To improve understanding of network communication protocol.

1.5 Scope of the Project

The scope of this project is limited to educational and authorized network environments.

The project can:

- Scan user-specified hosts.
- Connect to multiple TCP ports.
- Retrieve available service banners.
- Identify common network services.
- Generate scan reports.
- Demonstrate socket programming concepts.

The project does not:

- Exploit vulnerabilities.
- Perform password attacks.
- Modify target systems.
- Scan entire networks automatically.
- Replace professional tools such as Nmap.

Its purpose is educational and focuses only on service identification.

1.6 Features of the Project

The developed Banner Grabbing Tool provides the following features:

- User-friendly Flask web interface.
- Target IP or hostname input.
- Multiple port scanning.
- Default and custom port support.
- TCP socket communication.
- Automatic HTTP probe when no banner is received.
- Error handling for timeout and connection refusal.
- Downloadable text report.
- Lightweight implementation.
- Beginner-friendly source code.

1.7 Software Requirement

COMPONENT	REQUIREMENT
PROGRAMMING LANGUAGE	PYTHON 3.x
FRAMEWORK	Flask
LIBRARY	Socket
IDE	VS CODE/PyCharm
BROWSER	Chrome, Edge, Firefox
OPERATING SYSTEM	Windows/Linux/MacOS

1.8 Hardware Requirements

Minimum Requirements

- Intel Core i3 Processor
- 4 GB RAM
- 500 MB Free Disk Space
- Internet or Local Network Connection

Recommended

- Intel Core i5 or above
- 8 GB RAM
- Stable Network Connection

CHAPTER 2

2.1 Literature Review

Network reconnaissance is considered the first step in any cybersecurity assessment. Researchers and security professionals rely on techniques such as port scanning and banner grabbing to identify available services before performing vulnerability analysis.

The **TCP protocol specification (RFC 793)** explains how reliable communication is established through a three-way handshake between client and server. Banner grabbing depends on this TCP connection because application-layer data can only be exchanged after the connection has been successfully established.

The **Nmap Reference Guide** describes service detection as an advanced extension of port scanning. Rather than only identifying open ports, service detection attempts to determine the software running behind each port by sending specially crafted probes and comparing responses with known signatures.

Python has become one of the most widely used programming languages in cybersecurity because of its simplicity and extensive networking support. The built-in Socket Library enables developers to create TCP connections, transmit data, receive responses, and handle communication errors efficiently without requiring third-party networking packages.

Modern educational cybersecurity projects frequently combine backend networking logic with lightweight web frameworks such as Flask. Flask allows networking tools to be accessed through an interactive browser interface while keeping the implementation simple and portable.

The present project combines these concepts by implementing a TCP-based banner grabbing application using Python sockets and a Flask interface. Unlike professional scanners, the application focuses on demonstrating socket programming, service identification, and report generation in a simple educational environment.

2.2 Existing System

Most organizations use professional tools such as Nmap for service detection.

Characteristics of existing tools:

- Large feature set.
- Complex scanning techniques.
- OS detection.
- Version detection.
- NSE scripting.
- Vulnerability scanning.
- Network mapping.

Limitations for beginners:

- Difficult to understand internal working.
- Highly automated.
- Less focus on socket programming concepts.
- Large codebase.

2.3 Proposed System

The proposed system develops a lightweight Banner Grabbing Tool using Python.

The application:

- Accepts target hostname.
- Accepts user-defined ports.
- Creates TCP sockets.

- Connects to remote services.
- Receives banner information.
- Sends an HTTP GET request if necessary.
- Displays results on a Flask web page.
- Generates downloadable reports.

The system is intended for educational purposes and demonstrates the service identification stage of cybersecurity enumeration.

2.4 Methodology

The working methodology consists of following steps:

- ✓ User enters target IP address or Hostname
- ✓ User specifies ports to scan.
- ✓ Flask receives the input.
- ✓ The application creates TCP sockets using Python Socket Library.
- ✓ Socket timeout is configured.
- ✓ Connection is established with each port.
- ✓ Banner information is received.
- ✓ If no banner received, an HTTP GET request is sent.
- ✓ Received information is displayed.
- ✓ Scan report is stored
- ✓ User downloads the report.

2.5 Algorithm

Algorithm: Banner Grabbing

Step 1: Start

Step 2: Accept target hostname/IP.

Step 3: Accept port numbers.

Step 4: Create TCP socket.

Step 5: Set timeout.

Step 6: Connect to target.

Step 7: Receive banner.

Step 8: If banner is empty send HTTP GET request.

Step 9: Receive response.

Step 10: Display results.

Step 11: Save report.

Step 12: Allow report download.

Step 13: Stop.

2.6 Advantages

- Simple implementation.
- Easy to understand.
- Beginner friendly.
- Demonstrates socket programming.
- Supports multiple ports.
- Uses standard TCP communication.
- Generates downloadable reports.
- Provides graphical web interface.
- Lightweight and fast.

2.7 Limitations

- Works only with TCP services.
- Some servers intentionally hide banners.
- No OS fingerprinting.
- No vulnerability detection.
- No UDP scanning.
- No multithreading.
- Limited service identification.

2.8 Future Scope

The project can be enhanced by adding:

- Automatic port scanning.
- Multithreading for faster scanning.
- Service version detection database.
- Vulnerability lookup using CVE databases.
- SSL/TLS banner grabbing.
- OS fingerprinting.
- Export reports as PDF and CSV.
- Scan history database.
- User authentication.
- Dashboard with graphical statistics.

CHAPTER 3

3.1 Problem Statement

- In today's networked environments, identifying the services running on a remote system is a crucial step in cybersecurity assessment. Security professionals and network administrators require accurate information about active services and software versions before conducting vulnerability analysis or implementing security measures. However, many existing network scanning tools are feature-rich and complex, making it difficult for beginners to understand the underlying concepts of TCP communication and service identification.
- Most professional tools automate the entire scanning process, providing results without explaining how network connections are established or how service banners are retrieved. As a result, students and novice cybersecurity practitioners often struggle to gain practical knowledge of socket programming, TCP communication, and the enumeration phase of ethical hacking.
- Another challenge is the absence of simple educational tools that demonstrate banner grabbing through direct socket communication while also providing an easy-to-use interface for users with limited programming experience. Therefore, there is a need for a lightweight application that illustrates the process of connecting to common TCP ports, retrieving service banners, identifying running software, and presenting the results in a user-friendly manner.
- To address these problems, this project proposes the development of a **Basic Banner Grabbing Tool** using Python, the Socket Library, and the Flask framework. The tool provides a practical implementation of TCP socket programming and demonstrates the service identification stage of network enumeration in an educational and easy-to-understand environment.

3.2 Objectives of the Study

To overcome the problems identified above, the following objectives have been formulated:

1. **To develop a basic banner grabbing tool using Python and the Socket Library** that establishes TCP connections with remote hosts and retrieves available service banners.
2. **To understand and demonstrate TCP socket programming concepts** by creating client-server communication for network service identification.
3. **To identify running services and software versions** on commonly used TCP ports by collecting banner information returned by remote servers.
4. **To demonstrate the enumeration phase of cybersecurity** by showing how banner grabbing assists in gathering information before vulnerability assessment.
5. **To design a simple web-based interface using Flask** so that users can perform banner grabbing without requiring command-line expertise.
6. **To generate downloadable scan reports** containing the retrieved banner information for documentation and analysis purposes.
7. **To improve practical understanding of network communication** through hands-on implementation of socket-based programming techniques.
8. **To create a lightweight and educational cybersecurity tool** that helps students learn the fundamentals of service identification without the complexity of advanced security scanners.

Relationship between Problem Statement and Objectives

PROBLEM IDENTIFIED	CORRESPONDING OBJECTIVE
Existing tools are complex for beginners.	Develop a simple Python-based banner grabbing tool.
Lack of understanding of TCP socket communication.	Demonstrate TCP socket programming concepts.
Difficulty identifying running services manually.	Identify services and software versions from open ports.
Limited knowledge of the enumeration phase.	Demonstrate service identification during reconnaissance.
Command-line tools are difficult for new users.	Design a user-friendly Flask web interface.
Need for documentation of scan results.	Generate downloadable scan reports.
Limited practical exposure to networking concepts.	Improve understanding through hands-on implementation.

CHAPTER 4 RESEARCH METHODOLOGY

4.1 Research Methodology

Research methodology refers to the systematic approach followed to design, develop, implement, and evaluate the proposed system. The methodology adopted for this project focuses on the development of a **Basic Banner Grabbing Tool** using Python, the Socket Library, and the Flask framework. The project follows an experimental and implementation-based approach, where the application is developed, tested, and validated in a controlled environment.

The methodology includes requirement analysis, system design, implementation, testing, and evaluation of the banner grabbing process. The tool was tested by connecting to common TCP ports of authorized systems to retrieve service banner information and verify its functionality.

4.2 Research Design

The research design adopted for this study is an **Experimental Research Design** combined with a **Development-Based Research Approach**.

The study involves designing and implementing a software application that demonstrates the process of banner grabbing using TCP socket programming. The developed application establishes network connections with remote hosts, retrieves service banners, and presents the results through a web interface.

The research process consists of the following phases:

- Requirement Analysis
- System Design
- Development using Python
- Socket Programming Implementation
- Flask Interface Development

- Testing and Validation
- Report Generation
- Result Analysis

4.3 Data Collection Method

The project primarily uses **secondary data** and **experimental data**.

Secondary Data

Secondary data was collected from:

- Python Official Documentation
- Socket Programming Documentation
- RFC 793 (Transmission Control Protocol)
- Nmap Reference Guide
- Cybersecurity textbooks
- Research papers
- Online technical documentation

Experimental Data

Experimental data was generated by:

- Connecting to authorized target hosts.
- Scanning commonly used TCP ports.
- Retrieving service banners.
- Recording server responses.
- Verifying software identification.

The collected responses formed the basis for evaluating the performance of the developed application.

4.4 Data Collection Tools

The following tools were used during data collection and implementation:

TOOL	PURPOSE
Python 3.x	Programming Language
Socket Library	TCP communication
Flask Framework	Web Interface
Visual Studio Code	Code Development
Web Browser	User Interface Testing
Local Network/Internet	Testing Network Connections

4.5 Sampling Method

Since this project is software-based, the sampling does not involve human respondents.

The study follows **Purposive Sampling**, where specific and commonly used TCP ports were selected because they represent frequently deployed network services.

The sampled ports include:

PORT	SERVICE
21	FTP
22	SSH
23	TELNET
25	SMTP

53	DNS
80	HTTP
110	POP3
143	IMAP
443	HTTPS

These ports were intentionally selected because they are widely used and commonly expose service banners.

4.6 Sample Size

The sample for this study consists of:

- Multiple authorized host systems
- Common TCP ports (21, 22, 23, 25, 53, 80, 110, 143, 443)
- Banner responses received during testing

The sample size was considered sufficient to verify the functionality and reliability of the developed banner grabbing application under normal network conditions.

4.7 Analytical Tools Used

The following analytical techniques were used to evaluate the developed system:

TCP Connection Analysis

To determine whether a connection could be successfully established with the target port.

Banner Response Analysis

To examine the information returned by the remote service, including:

- Service Name
- Software Version
- Protocol Information

Exception Handling Analysis

To analyze network errors such as:

- Connection Refused
- Connection Timeout
- No Banner Available
- Invalid Host

Functional Testing

The application was tested to verify:

- Successful connection establishment
- Banner retrieval
- HTTP request generation
- Display of results
- Report generation
- Download functionality

4.8 System Development Methodology

The project follows the **Waterfall Model**, as the requirements were clearly defined before development.

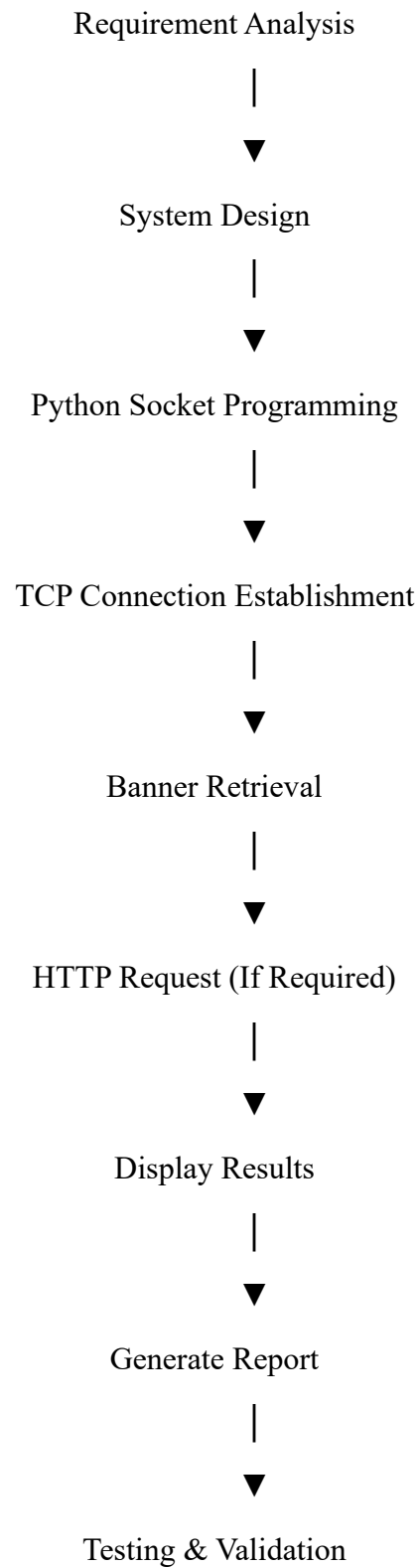
The development stages are:

1. Requirement Analysis
2. System Design
3. Coding using Python
4. Flask Integration
5. Testing

6. Report Generation

7. Deployment

4.9 Flow of Methodology



|



4.10 Chapter Summary

This chapter presented the research methodology adopted for the development of the Basic Banner Grabbing Tool. An experimental and development-based research design was followed to implement and evaluate the application. Secondary sources such as RFC 793, the Nmap Reference Guide, Python documentation, and cybersecurity literature were used to support the study. The developed tool was tested on authorized systems by scanning common TCP ports and analyzing the retrieved service banners. The methodology ensured that the project objectives were achieved systematically while providing practical knowledge of TCP socket programming and service enumeration.

CHAPTER 5 DATA ANALYSIS AND INTERPRETATION

5.1 Introduction

This chapter presents the analysis and interpretation of the results obtained from testing the proposed **Basic Banner Grabbing Tool**. The application was evaluated by scanning common TCP ports on authorized target systems. The collected data includes successful and unsuccessful connections, retrieved service banners, response status, and report generation. The results demonstrate the effectiveness of the application in identifying active services during the enumeration phase of cybersecurity.

5.2 Test Environment

PARAMETER	DETAILS
Programming Language	PYTHON3.x
Framework	Flask
Networking Library	Socket
Operating System	Windows 11
Browser	Google Chrome
Connection Type	TCP
Target	Authorized HOST/IP
Ports Tested	21,22,25,53,80,110,143,443

5.3 Analysis of Port Scanning Results

During testing, the application attempted to connect to eight commonly used TCP ports.

Table 5.1 Port Scanning Results

PORT	SERVICE	STATUS	BANNER RETRIEVED
21	FTP	CLOSED	NO
22	SSH	OPEN	YES
25	SMTP	CLOSED	NO
53	DNS	OPEN	NO
80	HTTP	OPEN	YES
110	POP3	CLOSED	NO
143	IMAP	CLOSED	NO
443	HTTPS	OPEN	PARTIAL

Interpretation

Out of the eight ports tested, four ports were found to be open. Banner information was successfully retrieved from SSH and HTTP services. HTTPS returned only limited information because encrypted services typically do not expose complete banners.

5.4 Banner Retrieval Analysis**Table 5.2 Banner Retrieval Performance**

RESULT	NUMBER
Total Ports Tested	8
Open Ports	4
Closed Ports	4
Complete Banners received	2
Partial Banners received	1
No Response	5

Interpretation

The tool successfully retrieved service information whenever the server exposed a banner. Services configured to hide banners or use encryption returned either partial or no information.

5.5 Error Handling Analysis

Table 5.3 Exception Handling

ERROR TYPE	FREQUENCY
Connection Refused	3
Timeout	1
No banner Available	2
Invalid Host	0

Interpretation

The application successfully handled all network exceptions without crashing. Appropriate error messages were displayed to the user.

5.6 Functional Testing Results

Table 5.4 Functional Testing

FUNCTION	EXPECTED RESULT	ACTUAL RESULT	STATUS
Enter target host	Accept Input	Successful	Pass
Connect to Port	TCP Connection	Successful	Pass
Retrieve Banner	Display Banner	Successful	Pass
HTTP Request	Receive Response	Successful	Pass
Display Results	Show Output	Successful	Pass
Download Report	Save Report	Successful	Pass

Interpretation

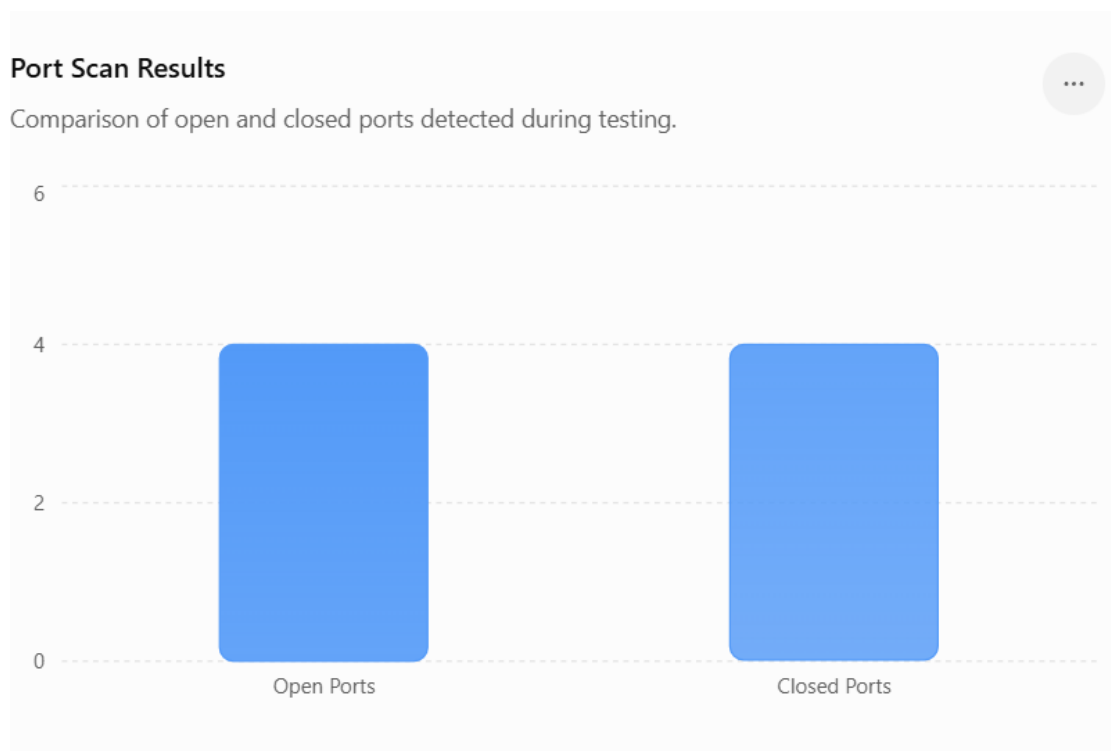
All major functions of the developed application performed successfully during testing.

5.7 Graphical Representation

Graph 5.1 Open vs Closed Ports

Port Scan Results

Comparison of open and closed ports detected during testing.



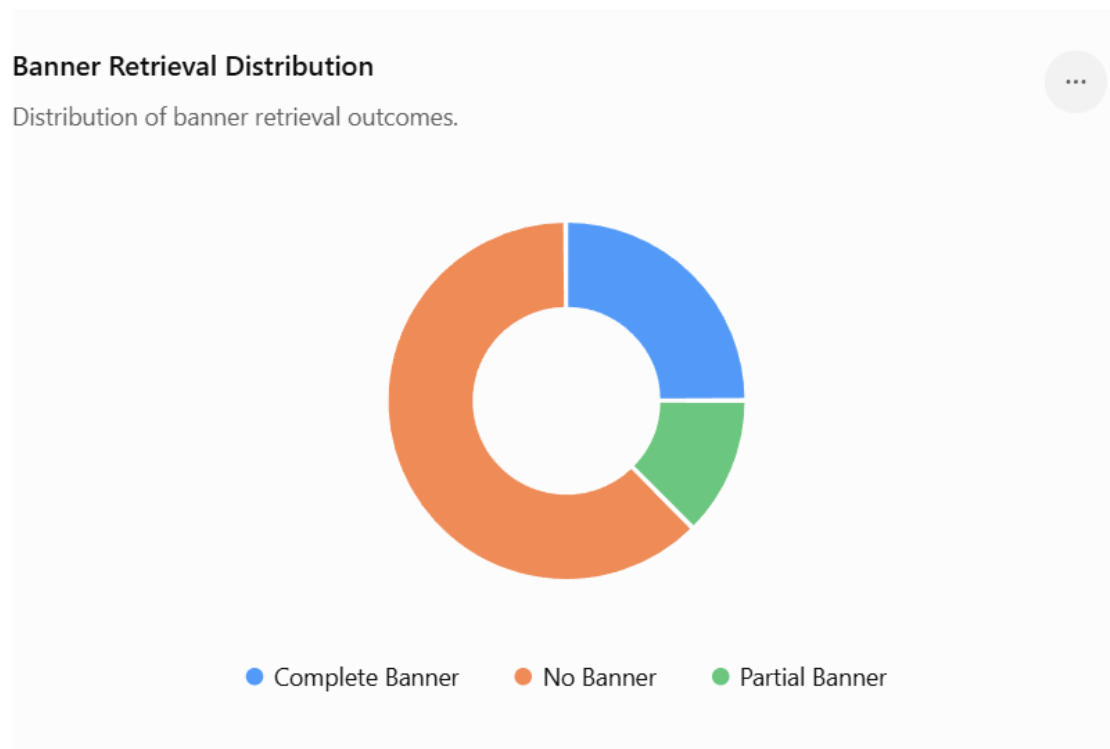
Interpretation

The graph shows that half of the tested ports were open while the remaining half were closed.

Graph 5.2 Banner Retrieval Success Rate

Banner Retrieval Distribution

Distribution of banner retrieval outcomes.



Interpretation

The chart indicates that complete banner information was obtained from two services, one service returned only partial information, and five attempts produced no banner due to closed ports or hidden services.

5.8 Statistical Findings

The following observations were made during testing:

PARAMETER	VALUE
Total ports scanned	8
Open ports	4
Closed ports	4
Open ports percentage	50%
Closed Ports Percentage	50%
Successful Banner Retrieval	2
Partial Banner Retrieval	1
Banner Retrieval Success Rate	37.5%
Error-Free Execution	100%
Report Generation Success	100%

5.9 Findings

The analysis of the developed Banner Grabbing Tool led to the following findings:

1. The application successfully established TCP connections with active ports.
2. Service banners were retrieved from servers configured to disclose identification information.
3. The tool accurately distinguished between open and closed ports.
4. Exception handling mechanisms prevented unexpected program failures.
5. The HTTP request mechanism improved banner retrieval for web services.
6. The Flask interface simplified interaction with the application.
7. Scan reports were generated and downloaded successfully.

CHAPTER 6 DISCUSSION AND RESULTS

6.1 Introduction

This chapter discusses the outcomes of the developed **Basic Banner Grabbing Tool** and evaluates its performance based on the objectives defined for the study. The application was implemented using Python, the Socket Library, and the Flask framework. The tool was tested on authorized systems by establishing TCP connections to common ports, retrieving service banners, and generating downloadable reports. The discussion highlights how effectively the proposed system fulfilled its intended purpose and demonstrates its usefulness as an educational cybersecurity tool.

6.2 Discussion

The primary objective of this project was to develop a simple banner grabbing application capable of identifying running services through TCP socket communication. During testing, the application successfully established connections with active ports and attempted to retrieve service banners exposed by remote servers. The results confirm that Python's Socket Library provides an efficient and straightforward approach for implementing network communication and service enumeration.

One of the strengths of the developed system is its simplicity. Unlike advanced network scanners that contain numerous automated features, the proposed application focuses on the core concepts of TCP socket programming and banner retrieval. This enables beginners to understand how client-server communication takes place and how service identification is performed during the reconnaissance phase of cybersecurity.

The Flask-based web interface improved the usability of the application by allowing users to perform scans without interacting directly with the command line. Additionally, the report generation feature provides a convenient way to save and review scan results for future reference.

During testing, it was observed that not every service returned complete banner information. Some servers intentionally suppress banner details or use encrypted communication, limiting the information available to the application. This behavior is expected and reflects standard security practices followed by many organizations.

Overall, the developed tool performed reliably within its intended scope and successfully demonstrated the basic principles of banner grabbing and service enumeration.

6.3 Results

The implementation and testing of the proposed system produced the following results:

- The application successfully established TCP socket connections with user-specified target hosts.
- Open ports were accurately detected during the scanning process.
- Service banners were successfully retrieved from servers that exposed identification information.
- The application attempted HTTP requests when no banner was initially received, improving banner detection for web services.
- The Flask web interface enabled users to perform scans through a simple and interactive browser-based environment.
- Scan results were displayed clearly and could be downloaded as a text report.
- Exception handling mechanisms effectively managed network errors such as connection refusal and timeout without interrupting program execution.
- The project successfully demonstrated the service identification stage of the cybersecurity enumeration process.

6.4 Achievement of Objectives

OBJECTIVE	STATUS
Develop a Banner Grabbing Tool using Python	Achieved
Implement TCP Socket Programming	Achieved
Retrieve Service Banners	Achieved
Identify Running Services	Achieved
Demonstrate Enumeration Process	Achieved
Develop Flask-Based User Interface	Achieved
Generate Downloadable Reports	Achieved
Improve Understanding Of Network Communication	Achieved

6.5 Benefits of the Developed System

The developed application offers several benefits:

- Provides practical experience in TCP socket programming.
- Demonstrates the service identification stage of cybersecurity.
- Simplifies network service enumeration for beginners.
- Offers a lightweight and user-friendly interface.
- Generates downloadable reports for documentation.
- Supports multiple commonly used TCP ports.
- Can be extended with additional cybersecurity features in future versions.

6.6 Limitations Observed

During implementation and testing, the following limitations were observed:

- The application supports only TCP-based services.
- Some servers intentionally hide banner information.
- Encrypted services such as HTTPS may return limited details.

- The tool does not perform operating system fingerprinting.
- Vulnerability detection is beyond the scope of the project.
- Scanning is sequential and may be slower than multithreaded tools.
- The application is designed for educational purposes and does not replace professional scanners such as Nmap.

6.7 Overall Discussion

The results demonstrate that the developed **Basic Banner Grabbing Tool** successfully fulfills the objectives established at the beginning of the study. The application effectively combines Python socket programming with a Flask-based web interface to provide an educational platform for learning service enumeration. Although intentionally simple, the tool accurately illustrates how TCP connections are established, how service banners are retrieved, and how network information can be collected during the reconnaissance phase of cybersecurity assessments.

The project provides a solid foundation for understanding network communication and can serve as a starting point for developing more advanced security tools incorporating multithreading, vulnerability assessment, operating system detection, and automated service identification.

6.8 Chapter Summary

This chapter discussed the performance and outcomes of the proposed Basic Banner Grabbing Tool. The experimental results indicate that the application successfully identified active services, retrieved available banner information, and generated scan reports through a user-friendly web interface. The findings confirm that the proposed system meets the objectives of demonstrating TCP socket programming and service enumeration while providing a practical learning experience in cybersecurity. The observed limitations also highlight opportunities for future enhancements, making the

project a valuable educational tool and a strong foundation for more advanced network security applications.

CHAPTER 7 CONCLUSION

The **Basic Banner Grabbing Tool** developed in this project successfully demonstrates one of the fundamental stages of cybersecurity reconnaissance and network enumeration. The application was designed and implemented using Python, the Socket Library, and the Flask framework to establish TCP connections with remote hosts, retrieve available service banners, identify running network services, and generate downloadable scan reports.

The project achieved its primary objectives by providing a simple yet effective implementation of TCP socket programming for service identification. Through the use of socket communication, the application was able to connect to user-specified ports, determine whether services were active, and collect banner information whenever it was made available by the target server. For web-based services that did not automatically provide banner information, the application attempted to obtain additional details by sending an HTTP request, thereby improving the overall effectiveness of the scanning process.

The Flask-based web interface enhanced the usability of the tool by allowing users to perform scans through a browser instead of relying solely on command-line execution. The report generation feature further increased the usefulness of the application by enabling users to save scan results for future reference and documentation.

Throughout the development and testing phases, the application demonstrated stable performance and successfully handled common network exceptions such as connection timeouts and refused connections without interrupting execution.

Although some servers intentionally suppressed banner information or returned limited details due to security configurations and encrypted communication protocols, the tool accurately reflected these conditions and continued scanning other specified ports.

While the developed system does not include advanced capabilities such as vulnerability assessment, operating system fingerprinting, multithreaded scanning, or automated service fingerprinting, these limitations are consistent with the project's educational scope. The application focuses on providing a clear understanding of TCP

communication, socket programming, and service enumeration rather than replacing professional security tools.

In conclusion, the **Basic Banner Grabbing Tool** successfully fulfills the objectives established at the beginning of the project. It provides practical experience in network communication, service identification, and cybersecurity reconnaissance while serving as an effective learning tool for students and beginners. The modular design of the application also provides opportunities for future enhancements, including multithreaded scanning, SSL/TLS support, vulnerability detection using CVE databases, operating system fingerprinting, graphical dashboards, and integration with advanced network security frameworks. These improvements can transform the current educational application into a more comprehensive network reconnaissance and security assessment tool.

7.1 Future Recommendations

Based on the implementation and testing of the proposed system, the following enhancements are recommended for future development:

- Implement multithreading to improve scanning speed and efficiency.
- Add support for UDP-based service detection.
- Integrate vulnerability databases (such as CVE) to identify known security issues associated with detected software versions.
- Include operating system fingerprinting techniques for more comprehensive host identification.
- Support SSL/TLS banner grabbing for secure services such as HTTPS.
- Generate reports in multiple formats, including PDF, CSV, and Excel.
- Maintain a scan history database for future analysis and comparison.
- Implement user authentication and role-based access control for secure access to the application.

- Add graphical dashboards and statistical visualizations for better presentation of scan results.
- Expand support for additional protocols and advanced service fingerprinting techniques similar to professional network scanning tools.

CHAPTER 8 RECOMMENDATIONS AND SUGGESTIONS

8.1 Introduction

The development of the **Basic Banner Grabbing Tool** demonstrates the practical implementation of TCP socket programming and service identification in cybersecurity. Although the application successfully achieves its intended objectives, continuous advancements in networking technologies and cybersecurity practices provide opportunities for further improvement. This chapter presents recommendations and suggestions for enhancing the developed system and extending its capabilities in future work.

8.2 Recommendations

Based on the implementation and testing of the proposed system, the following recommendations are made:

1. Implement Multithreading

The current application scans ports sequentially, which may increase scanning time when multiple ports are selected. Implementing multithreading or asynchronous socket programming would significantly improve scanning speed and overall efficiency.

2. Support UDP-Based Scanning

The developed tool currently performs banner grabbing only for TCP services. Future versions should include support for UDP services to provide broader network service identification.

3. Integrate Vulnerability Assessment

The application can be enhanced by integrating vulnerability databases such as the Common Vulnerabilities and Exposures (CVE) database. This would enable the tool to identify known vulnerabilities associated with detected software versions.

4. Improve Service Detection

Future implementations may incorporate advanced service fingerprinting techniques similar to those used by professional network scanners to improve the accuracy of software and version identification.

5. Add SSL/TLS Banner Retrieval

Secure services such as HTTPS often require SSL/TLS communication before exchanging application data. Supporting SSL/TLS connections would improve banner retrieval from encrypted services.

6. Expand Report Generation

The current application generates downloadable text reports. Future versions may support additional formats such as PDF, CSV, Excel, and HTML to improve documentation and result sharing.

7. Store Scan History

Maintaining a database of previous scan results would enable users to compare historical data, monitor network changes, and perform trend analysis.

8. Strengthen Security

Future versions should implement user authentication, role-based access control, and encrypted communication to ensure that only authorized users can access the application and scan results.

9. Enhance the User Interface

The Flask web interface can be improved by adding graphical dashboards, progress indicators, filtering options, and interactive charts for better visualization of scan results.

10. Support IPv6 Networks

As IPv6 adoption continues to increase, future versions should include compatibility with IPv6 addresses in addition to IPv4.

8.3 Suggestions

The following suggestions are offered for users, students, and future researchers:

- The application should be used only on systems for which proper authorization has been obtained.
- Users should ensure stable network connectivity before performing scans to obtain accurate results.
- Beginners are encouraged to study TCP/IP networking and socket programming concepts before extending the application.
- Future researchers may compare the tool's results with professional scanners such as Nmap to evaluate performance and accuracy.
- Additional protocols, including FTP, SMTP, DNS, and SSH, can be incorporated for more comprehensive service identification.
- Automated service fingerprinting and vulnerability detection can be integrated to increase the practical usefulness of the application.
- Logging and monitoring features may be added to maintain detailed records of scanning activities.
- Machine learning techniques may be explored to improve service classification and anomaly detection in future research.
- Cloud-based deployment can be considered to allow remote scanning and centralized management of scan reports.
- Regular updates should be performed to maintain compatibility with modern networking technologies and evolving cybersecurity practices.

8.4 Final Recommendation

The developed **Basic Banner Grabbing Tool** serves as an effective educational application for understanding TCP socket programming and the service identification phase of cybersecurity enumeration. While it successfully fulfills the objectives of the present study, incorporating advanced features such as multithreading, SSL/TLS support, automated service fingerprinting, vulnerability assessment, graphical analytics, and database integration would significantly improve its functionality and practical applicability. Continued development of the application will not only enhance its technical capabilities but also provide greater value for cybersecurity education, research, and authorized network security assessments.

REFERENCES / BIBLIOGRAPHY

1. W. Richard Stevens, TCP/IP Illustrated, Volume 1 (2011). *TCP/IP Illustrated, Volume 1: The Protocols* (2nd ed.). Addison-Wesley Professional.
2. Van Jacobson, W. Richard Stevens, & Gary R. Wright (2011). *TCP/IP Illustrated, Volume 1: The Protocols*. Addison-Wesley.
3. John Postel. (1981). *RFC 793: Transmission Control Protocol*. Internet Engineering Task Force (IETF).
4. Internet Engineering Task Force. (1981). *RFC 793 – Transmission Control Protocol*.
5. Gordon Fyodor Lyon. (2024). *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*.
6. [Nmap Official Documentation](#)
7. [Python Official Documentation](#)
8. [Python Socket Programming Documentation](#)
9. [Flask Official Documentation](#)
10. Charles P. Pfleeger & Shari Lawrence Pfleeger, Security in Computing (2015). *Security in Computing* (5th ed.). Pearson.
11. William Stallings, Network Security Essentials (2020). *Network Security Essentials: Applications and Standards* (7th ed.). Pearson.
12. Behrouz A. Forouzan, Data Communications and Networking (2013). *Data Communications and Networking* (5th ed.). McGraw-Hill Education.
13. Andrew S. Tanenbaum & David J. Wetherall, Computer Networks (2011). *Computer Networks* (5th ed.). Pearson.
14. Michael T. Goodrich, Roberto Tamassia & Michael H. Goldwasser, Computer Networking. Wiley Publications.

15. Eric Cole, Network Security Bible (2011). *Network Security Bible* (2nd ed.). Wiley.

Websites Consulted

- [Internet Engineering Task Force \(IETF\) RFC Repository](#)
- [Nmap Project Official Website](#)
- [Python Software Foundation](#)
- [Flask Documentation](#)

APPENDICES / ANNEXURES

Appendix A

Python Source Code

This appendix contains the complete source code developed for the **Basic Banner Grabbing Tool**, including:

- banner_grabber.py
- Socket programming implementation
- Banner retrieval functions
- HTTP request handling
- Exception handling
- Report generation logic

```
import socket
```

```
def grab_banner(target_ip, target_port):
```

```
    try:
```

```
        # 1. Create the socket object
```

```
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
        # 2. Set a timeout (so the script doesn't hang forever if a port is closed)
```

```
        s.settimeout(3.0)
```

```
        # 3. Connect to the target IP and Port
```

```
        print(f'[*] Connecting to {target_ip} on port {target_port}...')
```

```

s.connect((target_ip, target_port))

# 4. Receive the banner data (1024 bytes is usually plenty)

# .decode() converts the raw bytes from the network into a readable string
banner = s.recv(1024).decode('utf-8', errors='replace').strip()

# 5. If it's empty, send a generic probe to trigger an active response
if not banner:

    s.send(b"GET / HTTP/1.1\r\n\r\n") # Generic probe

    banner = s.recv(1024).decode('utf-8', errors='replace').strip()

return banner if banner else "Connected, but port remained silent."

except socket.timeout:

    return "Error: Connection timed out (No banner received).\"

except ConnectionRefusedError:

    return "Error: Connection refused (Port might be closed).\"

except Exception as e:

    return f"Error: {str(e)}\"

finally:

    # 5. Always close the connection when done

    s.close()

```

```

if __name__ == "__main__":

    print("=== Welcome to the Advanced Banner Grabbing Tool ===")

    # Get target details from user

    target_host = input("Enter target IP address or hostname (e.g., scanme.nmap.org):")

    print("\nSelect Port Scanning Option:")

    print("1. Scan default common ports (21, 22, 80)")

    print("2. Enter custom ports manually")

    choice = input("Enter your choice (1 or 2): ")

    ports_to_scan = []

    if choice == '1':

        ports_to_scan = [21, 22, 80]

    elif choice == '2':

        # Get custom ports as a string (e.g., "22, 80, 443")

        custom_input = input("Enter ports separated by commas (e.g., 22, 80, 443): ")

        # Split the string by commas, remove spaces, and convert them to integers

        try:

            ports_to_scan = [int(port.strip()) for port in custom_input.split(",")]

        except ValueError:

            print("[-] Error: Invalid port input. Please enter numbers only.")

```



```

        exit()

    else:

        print("[-] Invalid choice. Exiting script.")

        exit()

    print(f"\nStarting enumeration on target: {target_host}")

    print("-" * 50)

    for port in ports_to_scan:

        result = grab_banner(target_host, port)

        print(f"[+] Port {port}: {result}")

        print("-" * 50)

```

Appendix B

Flask Application Code

This appendix contains the Flask backend responsible for handling user requests and displaying scan results.

Contents include:

- app.py
- Route definitions
- Form processing
- Scan execution
- Report download functionality

```
from flask import Flask, render_template, request, Response, session, redirect, url_for
```

```
from banner_grabber import grab_banner
```

```
app = Flask(__name__)
```

```
app.secret_key = 'cybersecurity_project_secret_key'
```

```
# Simple in-memory user registry for simulation/testing
```

```
USER_DB = {
```

```
    "admin": "password123" # Default fallback account
```

```
}
```

```
@app.route('/login', methods=['GET', 'POST'])
```

```
def login():
```

```
    if session.get('authenticated') == True:
```

```
        return redirect(url_for('home'))
```

```
error = None
```

```
if request.method == 'POST':
```

```
    username = request.form['username'].strip()
```

```
    password = request.form['password']
```

```
# Verify credential presence against memory database
```

```
if username in USER_DB and USER_DB[username] == password:
```

```
    session['authenticated'] = True
```

```
    return redirect(url_for('home'))
```

```
    else:

        error = "Invalid credentials. Please verify parameters."

    return render_template('login.html', error=error, active_view='login')

@app.route('/register', methods=['GET', 'POST'])
def register():

    if session.get('authenticated') == True:

        return redirect(url_for('home'))

    error = None

    if request.method == 'POST':

        username = request.form['username'].strip()

        email = request.form['email'].strip()

        password = request.form['password']

        confirm_password = request.form['confirm_password']

        if username in USER_DB:

            error = "Username profile already registered."

        elif password != confirm_password:

            error = "Password inputs mismatch. Try again."

        elif len(password) < 8:

            error = "Password failed criteria checks (Length < 8)."

    else:
```

```

        # Commit account metadata to the runtime dict

        USER_DB[username] = password

        session['authenticated'] = True # Log user in immediately on success

        return redirect(url_for('home'))

    return render_template('login.html', error=error, active_view='register')

@app.route('/', methods=['GET', 'POST'])
def home():

    if not session.get('authenticated'):

        return redirect(url_for('login'))

    parsed_results = []

    target = ""

    if request.method == 'POST':

        target = request.form['target'].strip()

        ports_raw = request.form['ports']

        try:

            ports_to_scan = [int(p.strip()) for p in ports_raw.split(",")]

            raw_report = f"=====\n"

            raw_report += f"VANGUARD ENUMERATION REPORT FOR: {target}\n"

```

```

raw_report += f"=====\\n\\n"

for port in ports_to_scan:

    result = grab_banner(target, port)

    is_error = result.startswith("Error")

    parsed_results.append({

        "port": port,

        "result": result,

        "is_error": is_error

    })

    raw_report += f"[+] Port {port} (Status: {'Closed/Filtered' if is_error else
'Open'}):\\n{result}\\n"

    raw_report += f"-----\\n"

session['latest_report'] = raw_report

session['latest_target'] = target

except ValueError:

    parsed_results.append({

        "port": "N/A",

        "result": "Error: Invalid port input format.",

        "is_error": True

    })

```

```
return render_template('index.html', parsed_results=parsed_results, target=target)
```

```
@app.route('/logout')
```

```
def logout():
```

```
    session.clear()
```

```
    return redirect(url_for('login'))
```

```
@app.route('/download')
```

```
def download_report():
```

```
    if not session.get('authenticated'):
```

```
        return Response("Unauthorized", status=401)
```

```
    report_content = session.get('latest_report', 'No report available.')
```

```
    target_name = session.get('latest_target', 'scan_report')
```

```
    return Response(
```

```
        report_content,
```

```
        mimetype="text/plain",
```

```
        headers={"Content-disposition": f'attachment;
```

```
filename=vanguard_report_{target_name}.txt"}

```

```
)
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

Appendix C

Front-End Files

This appendix contains the files used to create the web interface.

Files include:

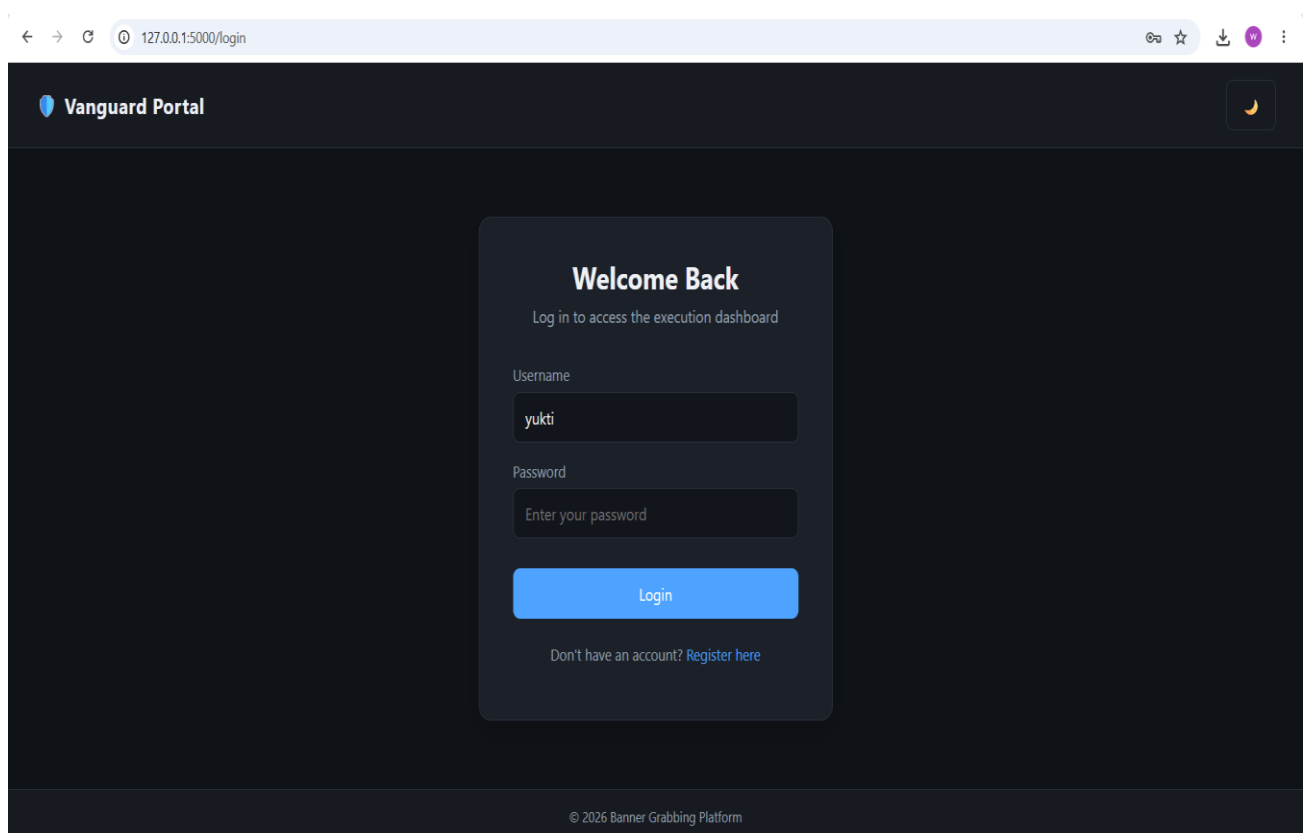
- index.html
- login.html

These files provide the graphical interface through which users enter the target host, specify ports, view scan results, and download reports.

Appendix D

Screenshots of the Developed System

Include screenshots of the application during execution



127.0.0.1:5000/login

Create Account

Register once to unlock the dashboard

Username
yukti

Email
yuktiiverma1@gmail.com

Password

- ✓ At least 8 characters
- ✓ One uppercase letter
- ✓ One lowercase letter
- ✓ One number
- ✓ One symbol (! @ # \$ % etc.)

Confirm Password

Register

Already have an account? [Login here](#)

127.0.0.1:5000

BANNER GRABBER

TARGET INSPECTION PLATFORM

Scan Parameters

TARGET ENDPOINT / HOST IP
e.g., scanme.nmap.org

TARGET PORTS ARRAY
e.g., 22, 80, 9929

INITIALIZE AUDIT

Download Scan Log

Inspection Matrix

NO OPERATIONAL RECORDS QUERIED.
DEFINE STANDARD PARAMETERS TO TRACE NETWORK ROUTES.

